Lesson 03 Demo 04

Deploying Kubernetes YAML with CodePipeline

Objective: To set up an automated deployment process that integrates AWS CodePipeline with Kubernetes, allowing for seamless application deployment to an EKS cluster

Tools required: AWS Management Console

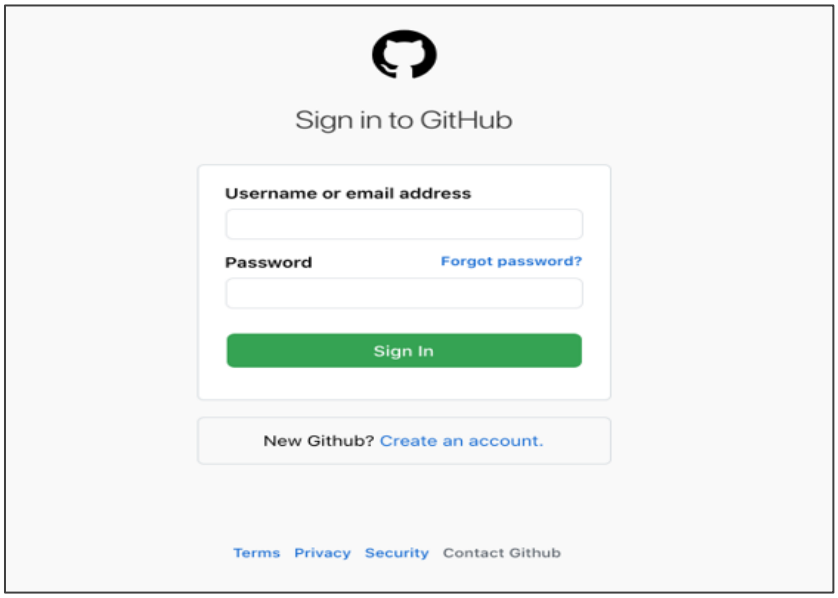
Prerequisites: Refer to Lesson 06 Demo 04 for creating an EKS cluster

Steps to be followed:

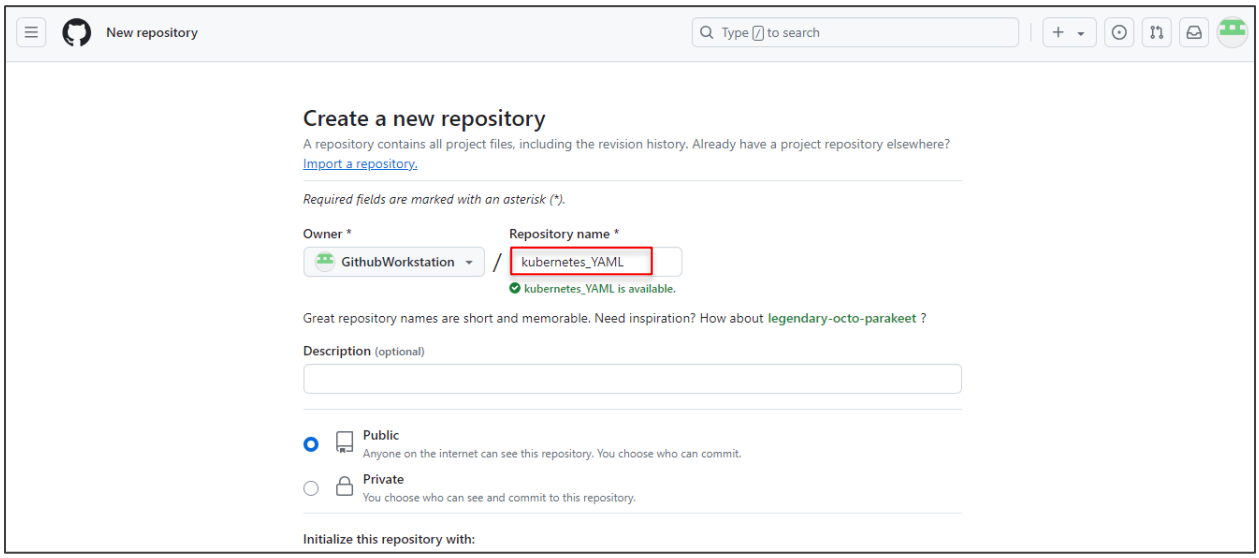
1. Create a GitHub repository
2. Create AWS CodePipeline to deploy manifests on EKS
3. Set up build commands

Step 1: Create a GitHub repository

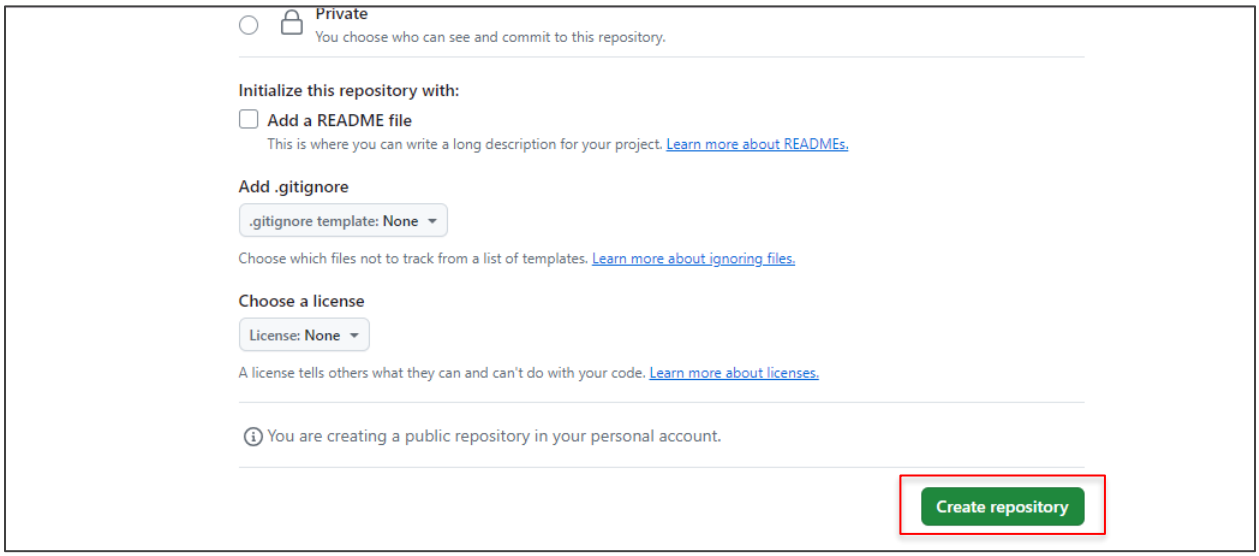
1.1 Navigate to GitHub and log in using your credentials



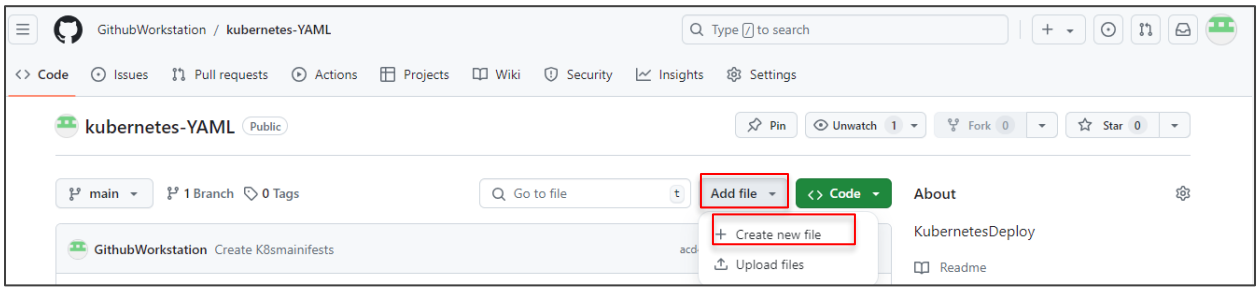
1.2 Provide a name for the repository, as shown in the screenshot below:



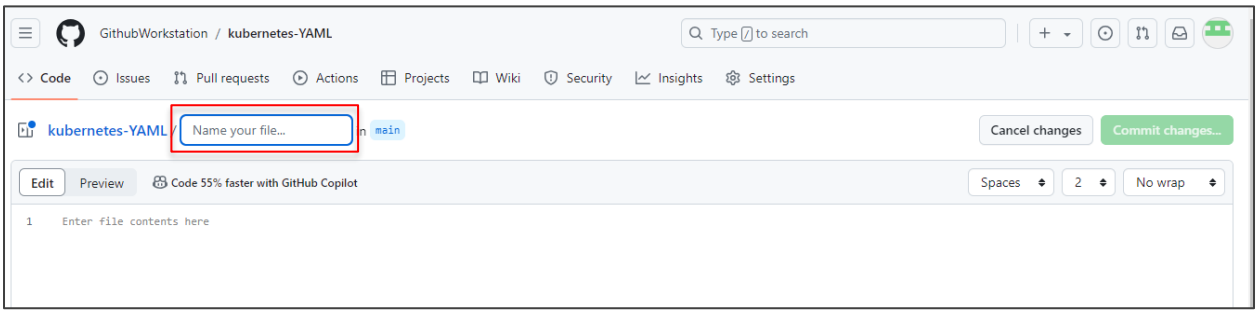
1.3 Click on **Create repository**



1.4 Click on **Add file**, then select **Create new file** as shown in the screenshot below:



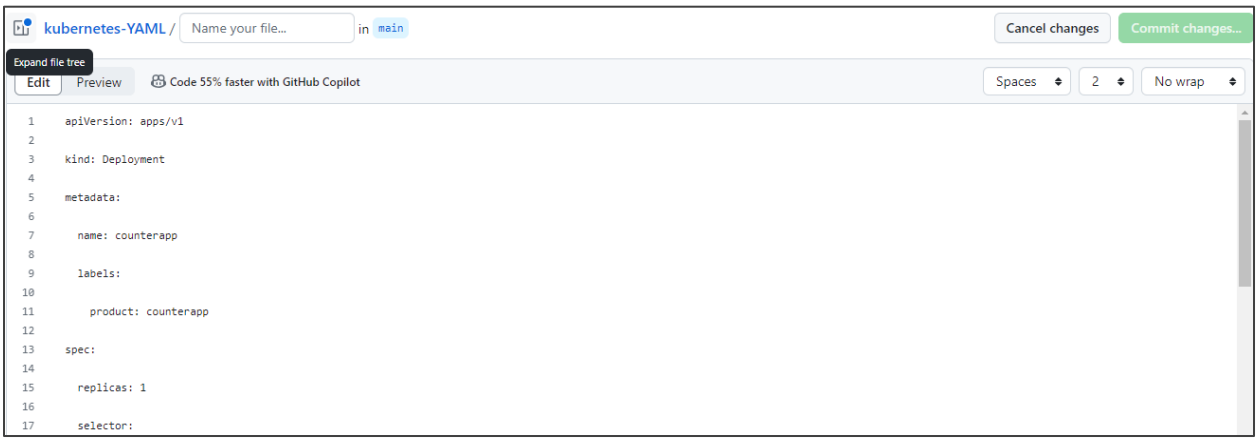
1.5 Name the file



1.6 Add the following YAML script inside the file and save it:

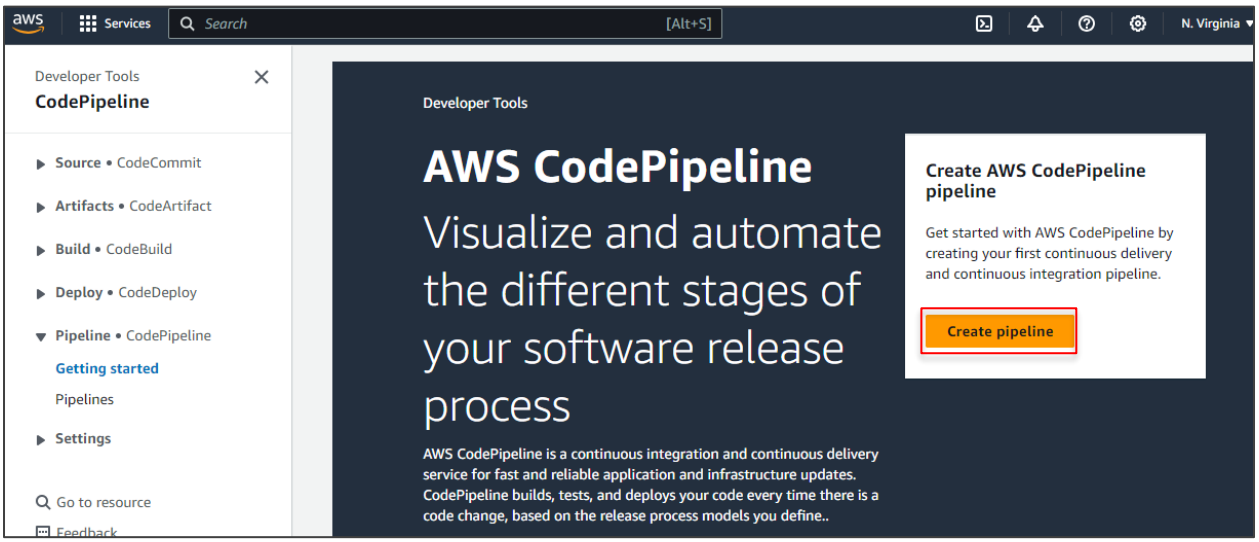
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: counterapp
  labels:
    product: counterapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: counterapp
  template:
    metadata:
      labels:
        app: counterapp
    spec:
      containers:
      - name: bootcamp-container
        image: jocatalin/kubernetes-bootcamp:v1
        ports:
```

```
- containerPort: 8080
resources:
  limits:
    cpu: "1"
    memory: "600Mi"
  requests:
    cpu: "500m"
    memory: "300Mi"
```

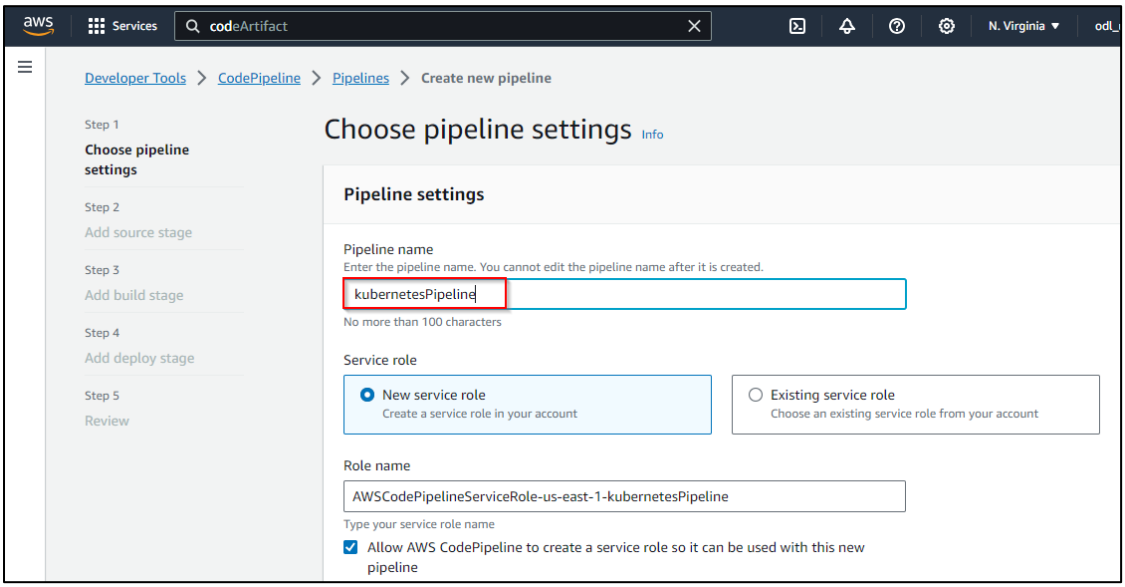


Step 2: Create AWS CodePipeline to deploy manifests on EKS

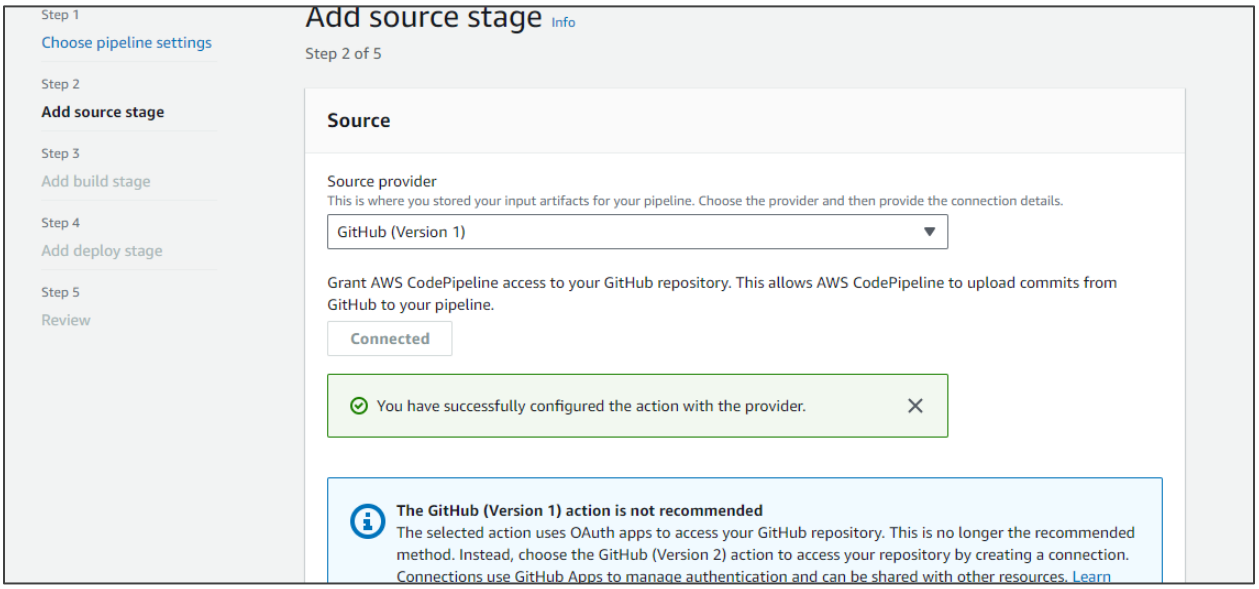
2.1 Navigate to the AWS management console, open CodePipeline, and then click on **Create pipeline**



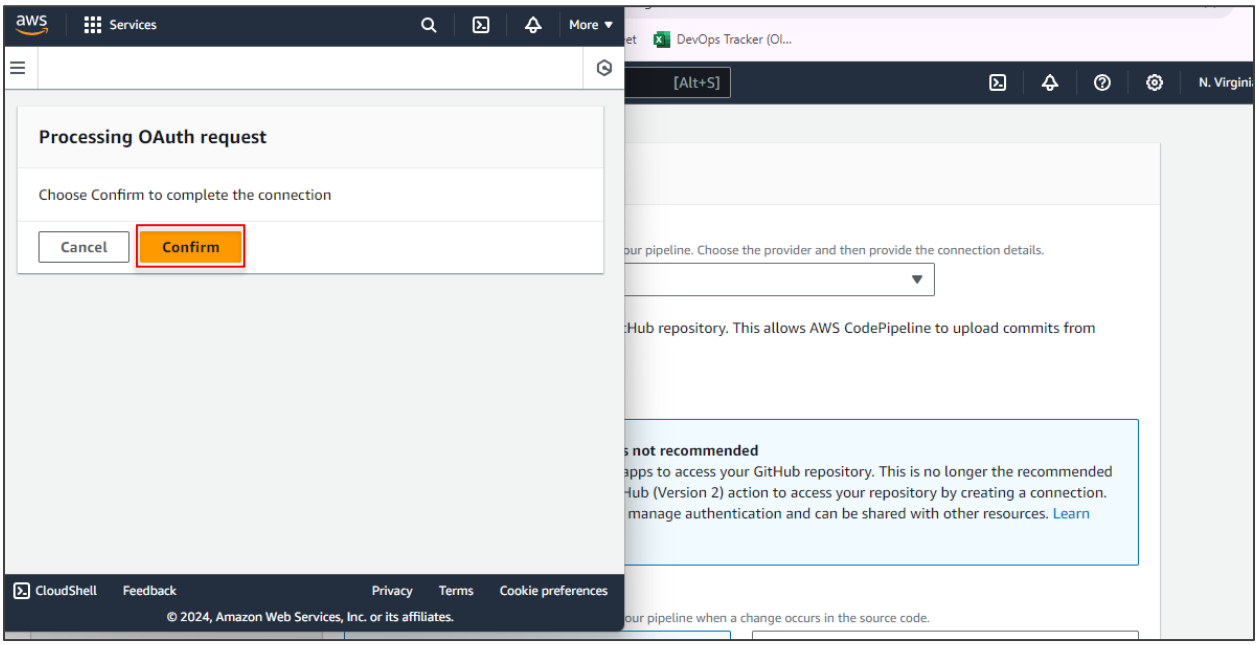
2.2 Name your pipeline **kubernetesPipeline**



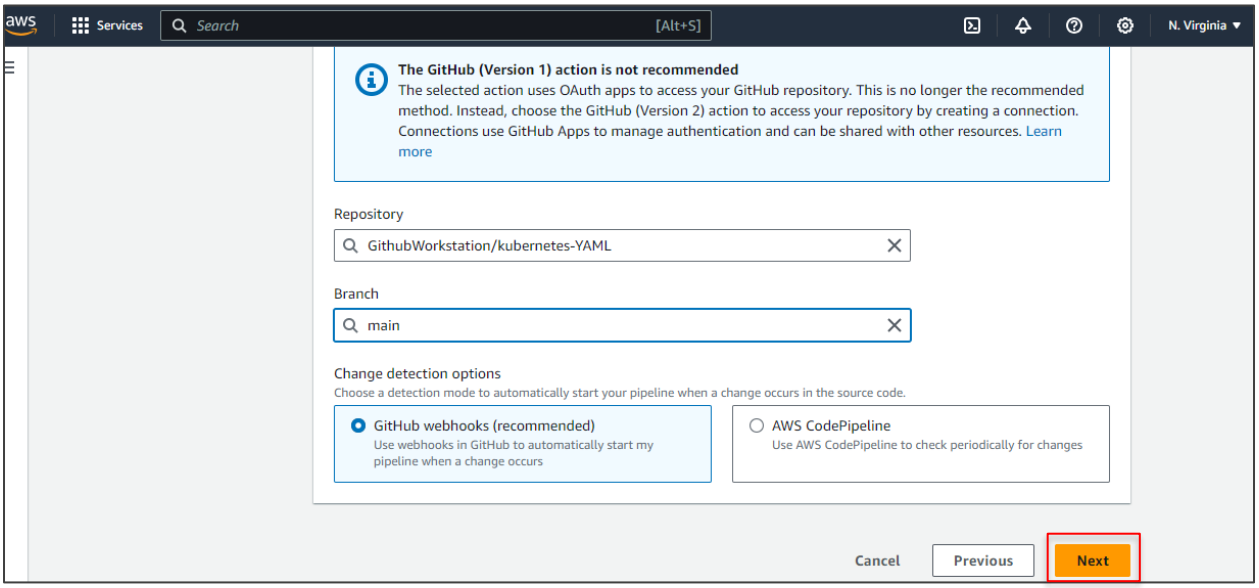
2.3 Select **GitHub (Version 1)** as the Source provider



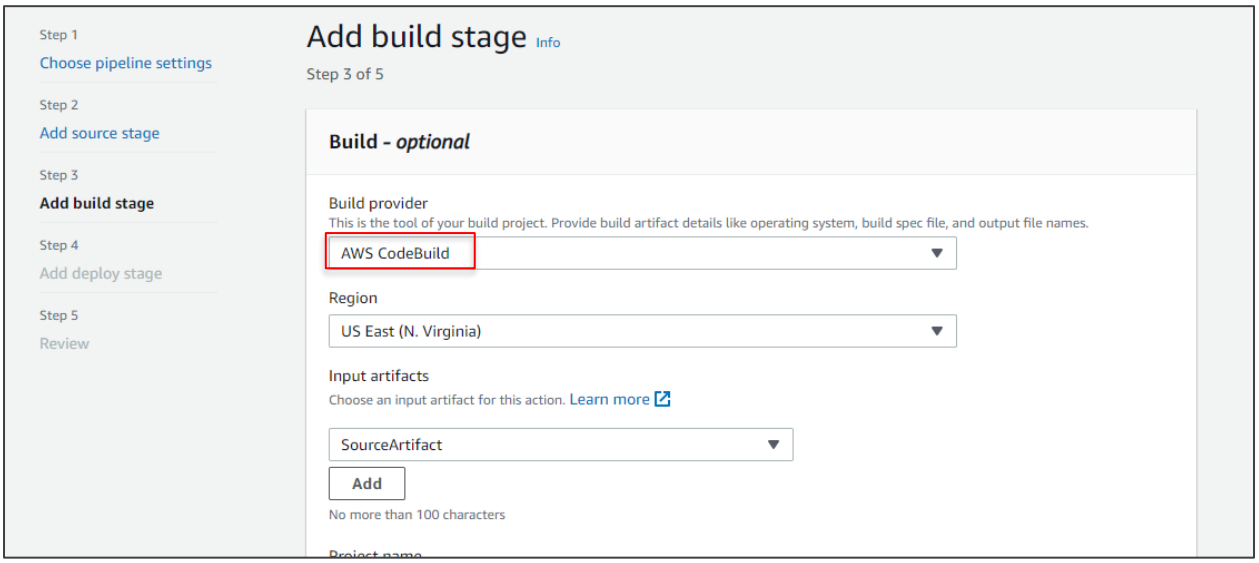
2.4 Click on **Confirm**



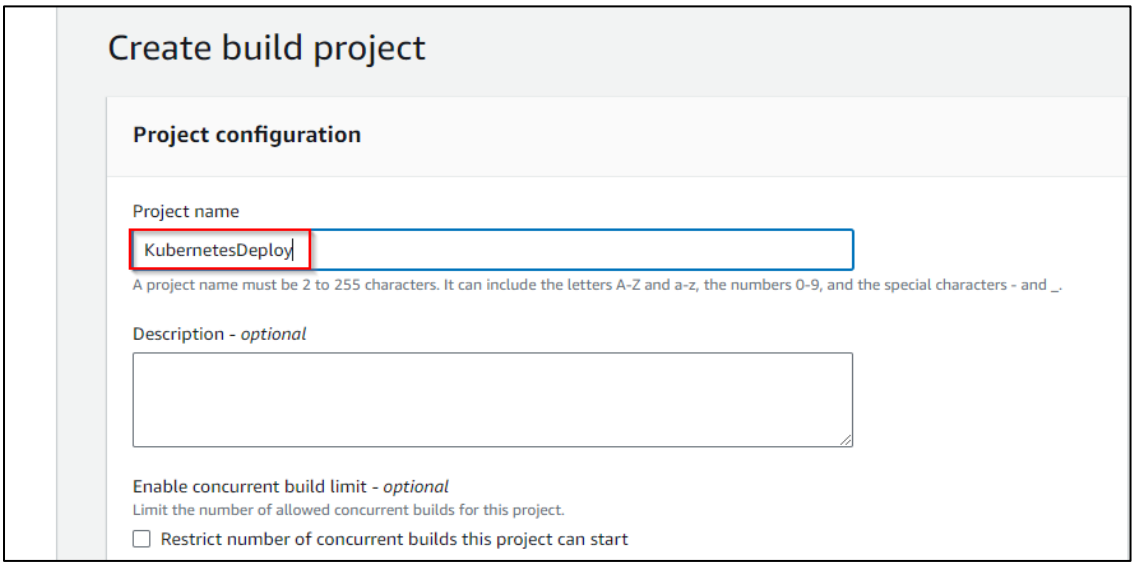
2.5 Choose the repository you created, mention the Branch as **main**, select **GitHub webhooks (recommended)** under Change detection options, and then click on **Next**, as shown in the screenshot below:



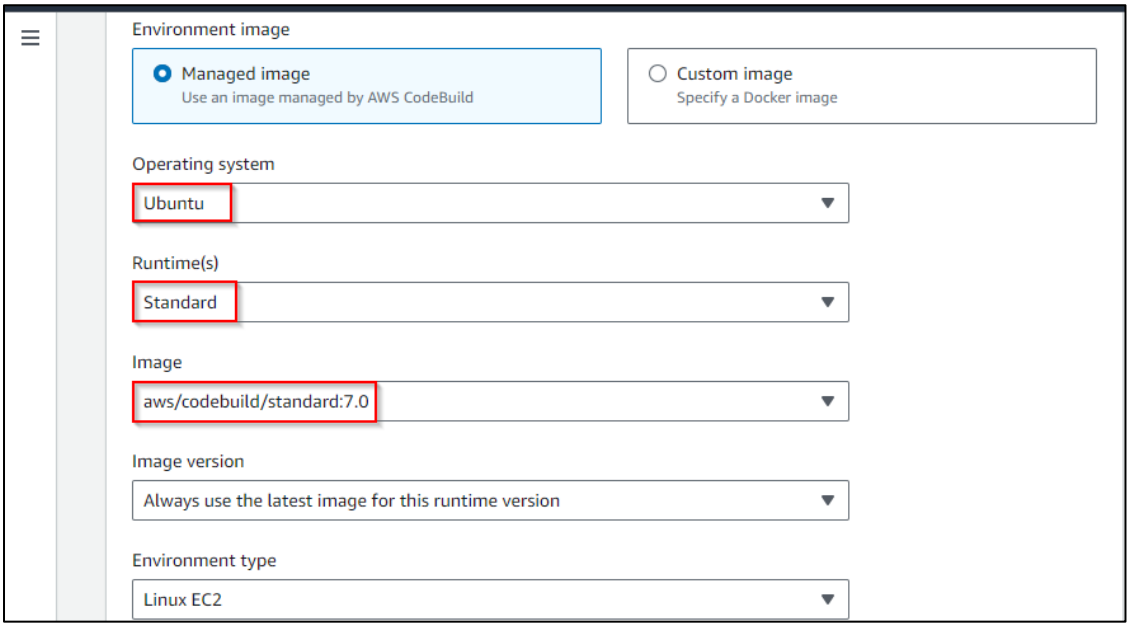
2.6 Choose **AWS CodeBuild** under Build provider as shown in the screenshot below:

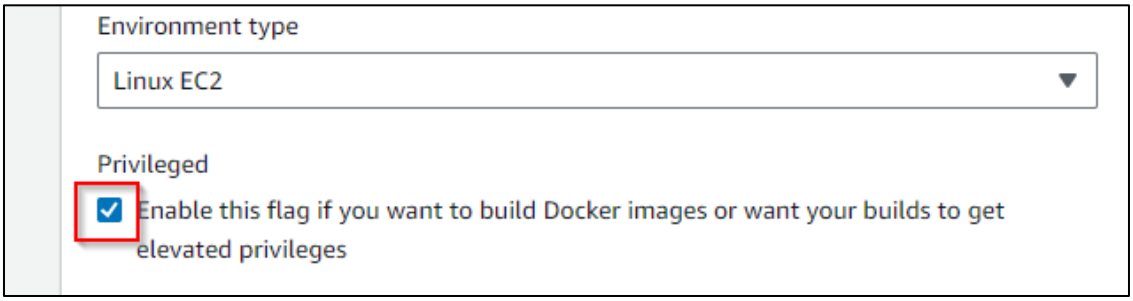


2.7 Name your project **KubernetesDeploy**

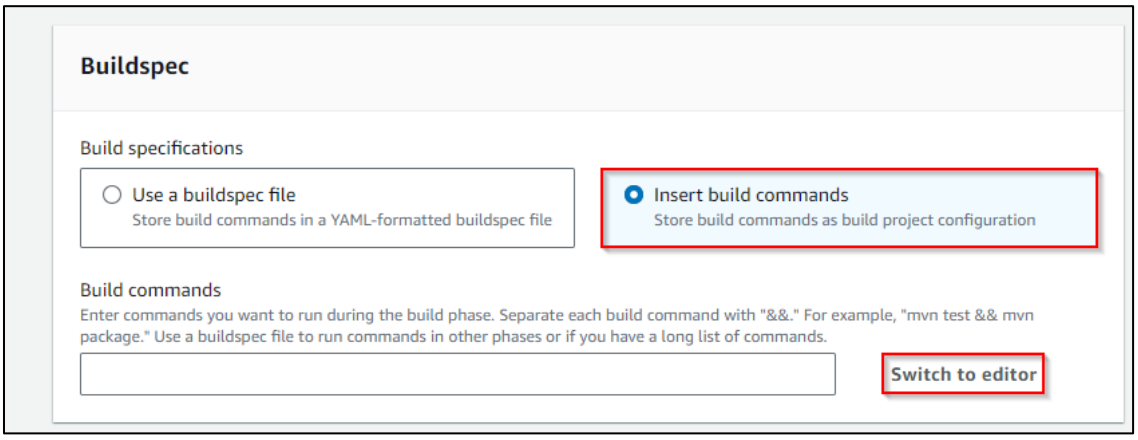


2.8 Select **Ubuntu** for the Operating system, **Standard** for Runtime(s), and **aws/codebuild/standard:7.0** for the Image. Make sure the **Privileged** option is checked.





2.9 Select **Insert build commands** for Buildspec and click **Switch to editor**



Step 3: Set up build commands

3.1 Add the following code into the build commands section:

```
version: 0.2
phases:
  build:
    commands:
      - echo "Update AWS CLI"
      - pip3 uninstall -y awscli
      - curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
      - unzip awscliv2.zip
      - sudo ./aws/install --update
      - export PATH=$PATH:/usr/local/bin
      - echo "Install Kubectl"
      - curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/1.21.2/2021-
07-05/bin/linux/amd64/kubectl
      - chmod +x ./kubectl
      - mkdir -p $HOME/bin && cp ./kubectl $HOME/bin/kubectl && export
PATH=$PATH:$HOME/bin
      - echo 'export PATH=$PATH:$HOME/bin' >> ~/.bashrc
      - kubectl version --client
  post_build:
    commands:
      - echo "Update Kube Config"
      - aws eks update-kubeconfig --name EKSCluster
      - aws sts get-caller-identity
      - echo "Apply changes to kube manifests"
```

- kubectl apply -f deployment.yaml

Build commands

```
3   build:
4     commands:
5       - echo "Update AWS CLI"
6       - pip3 uninstall -y awscli
7       - curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
8       - unzip awscliv2.zip
9       - sudo ./aws/install --update
10      - export PATH=$PATH:/usr/local/bin
11      - echo "Install Kubectl"
12      - curl -o kubectl https://amazon-eks.s3.us-west-2.amazonaws.com/1.21.2/2021-07-05/bin/linux
13      - chmod +x ./kubectl
14      - mkdir -p $HOME/bin && cp ./kubectl $HOME/bin/kubectl && export PATH=$PATH:$HOME/bin
15      - echo 'export PATH=$PATH:$HOME/bin' >> ~/.bashrc
16      - kubectl version --client
17  post_build:
18    commands:
19      - echo "Update Kube Config"
20      - aws eks update-kubeconfig --name EKSCluster
21      - aws sts get-caller-identity
22      - echo "Apply changes to kube manifests"
23      - kubectl apply -f deployment.yaml
24
```

Switch to single line

3.2 Click on **Continue to CodePipeline**

Group name

Stream name

S3

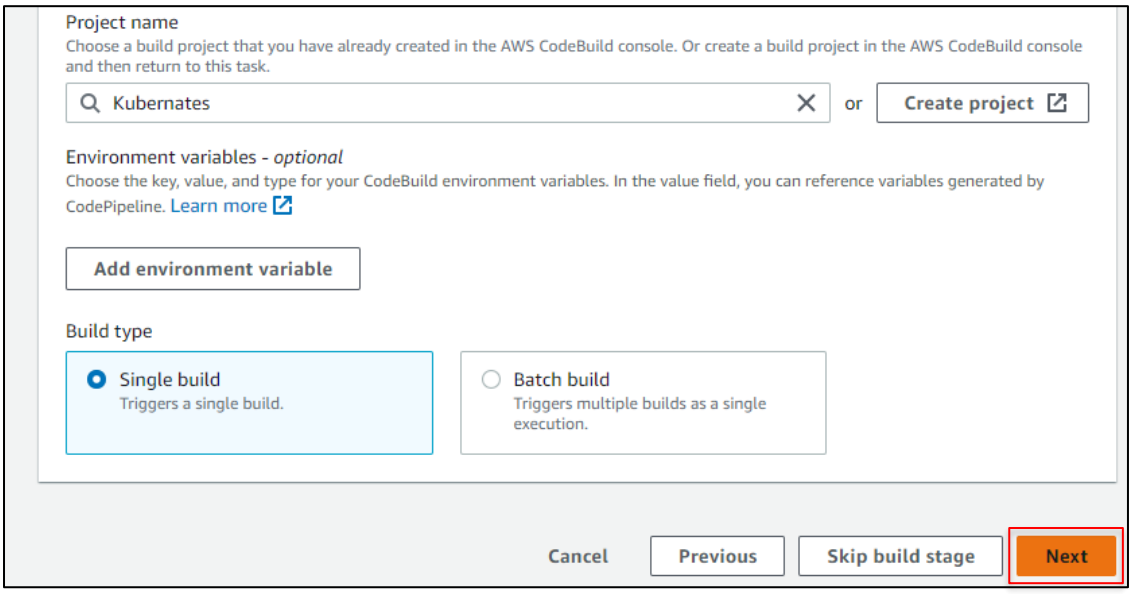
S3 logs - *optional*

Checking this option will upload build output logs to S3.

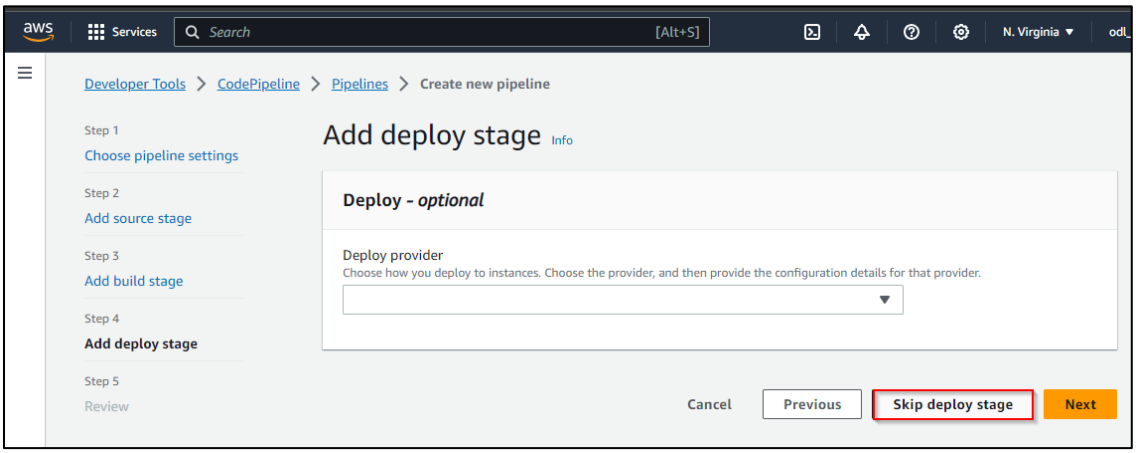
Cancel | **Continue to CodePipeline**

Note: If the CodePipeline is not working, create a new policy named **CodeBuildBasePolicy-KubernetesDeploy1-us-east-1-v2**, and attach **codebuild-KubernetesDeploy-role** and **codebuild-Ku-service-role** policies.

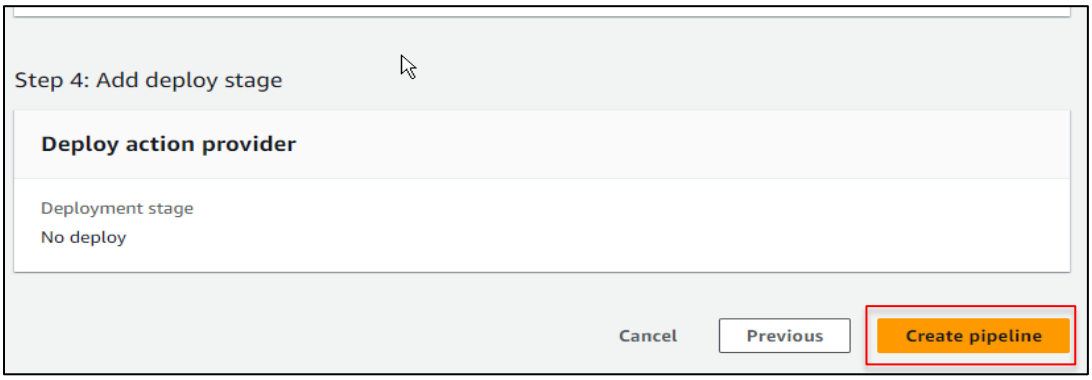
3.3 Click on **Next**



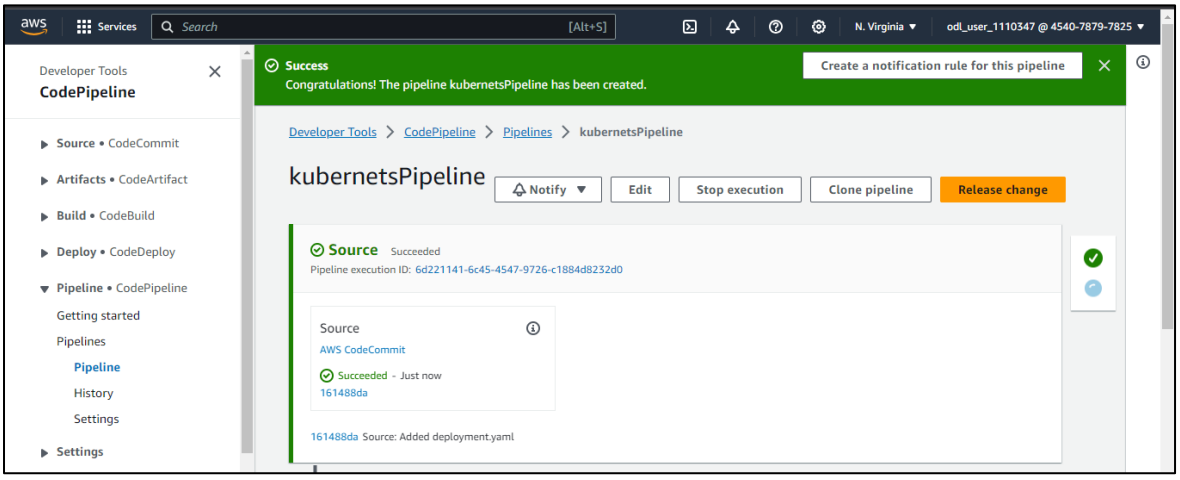
3.4 Click on **Skip deploy stage**



3.5 Conclude by clicking **Create pipeline**



You should see that the **kubernetesPipeline** has been successfully created.

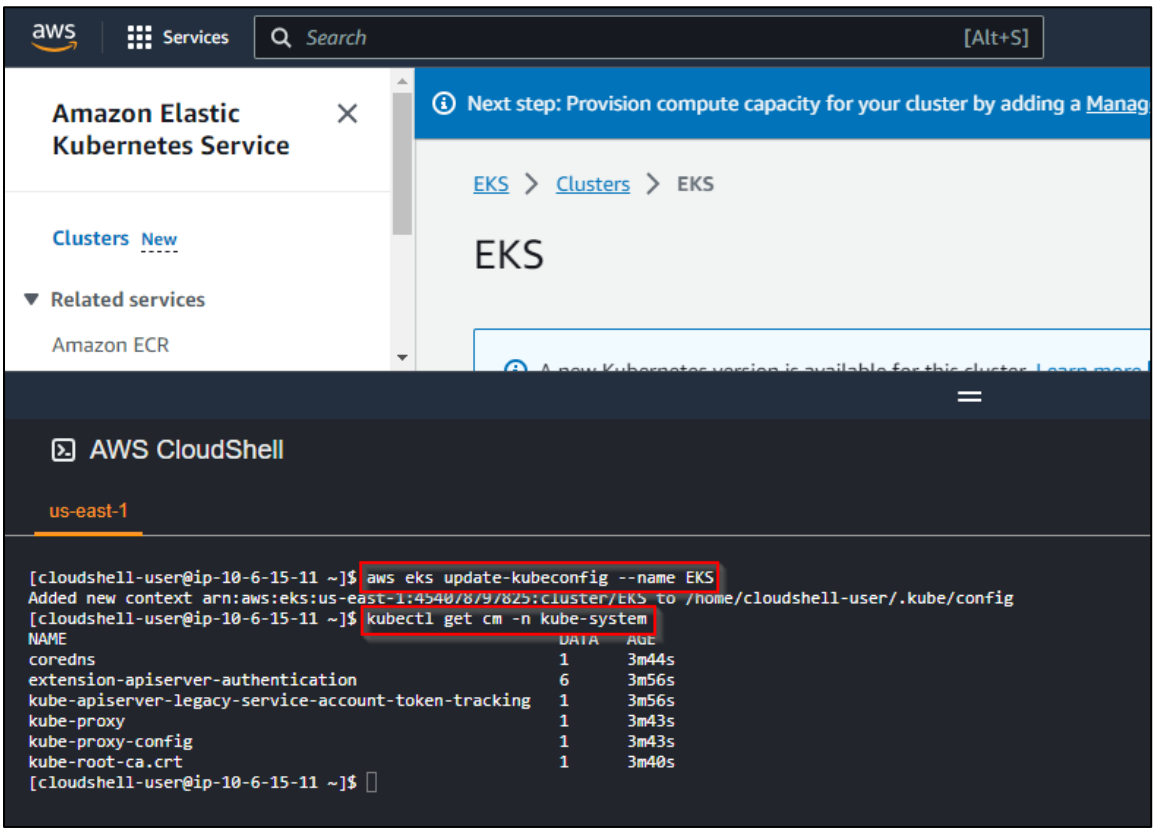


Note: If you encounter errors during the Build stage, follow the steps below.

3.6 Visit your EKS cluster and click the cloudShell icon. Enter the displayed commands to update Kubernetes:

aws eks update-kubeconfig --name EKS

kubectl get cm -n kube-system



Note: Refer to Lesson 06 Demo 04 for instructions on setting up an EKS cluster

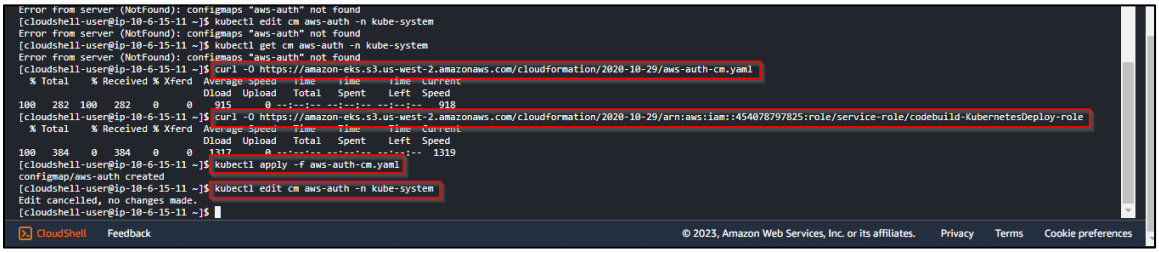
3.7 Enter the following commands to edit the Kubernetes system:

curl -O https://amazon-eks.s3.us-west-2.amazonaws.com/cloudformation/2020-10-29/aws-auth-cm.yaml

curl -O https://amazon-eks.s3.us-west-2.amazonaws.com/cloudformation/2020-10-29/arn:aws:iam::454078797825:role/service-role/codebuild-KubernetesDeploy-role

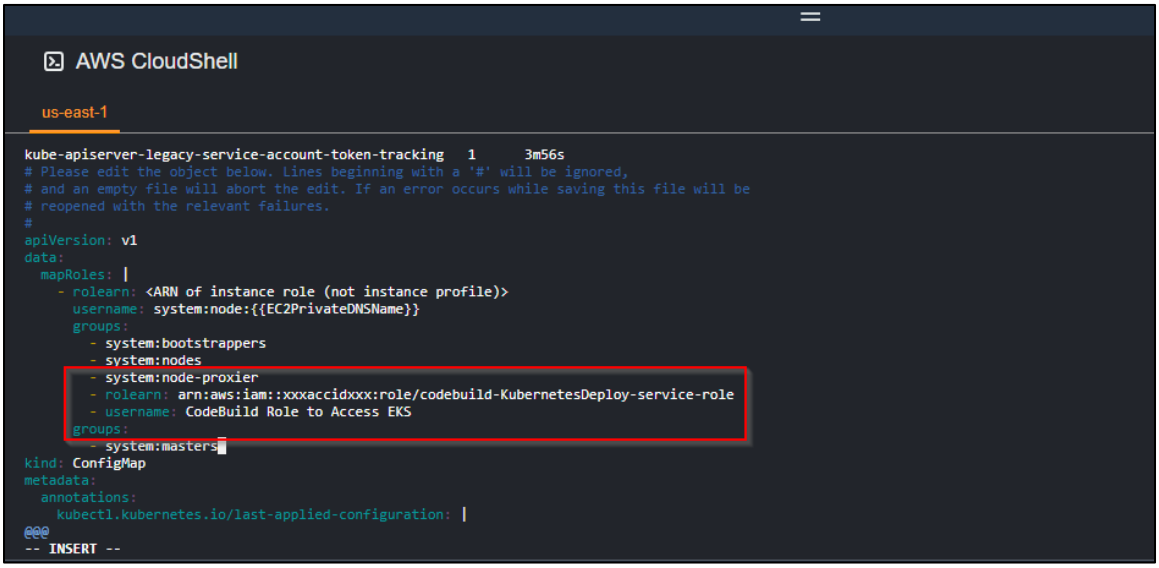
Modify the **aws-auth-cm.yaml** to include the correct ARN for your worker node IAM role, then apply the configuration:

kubectl apply -f aws-auth-cm.yaml
kubectl edit cm aws-auth -n kube-system



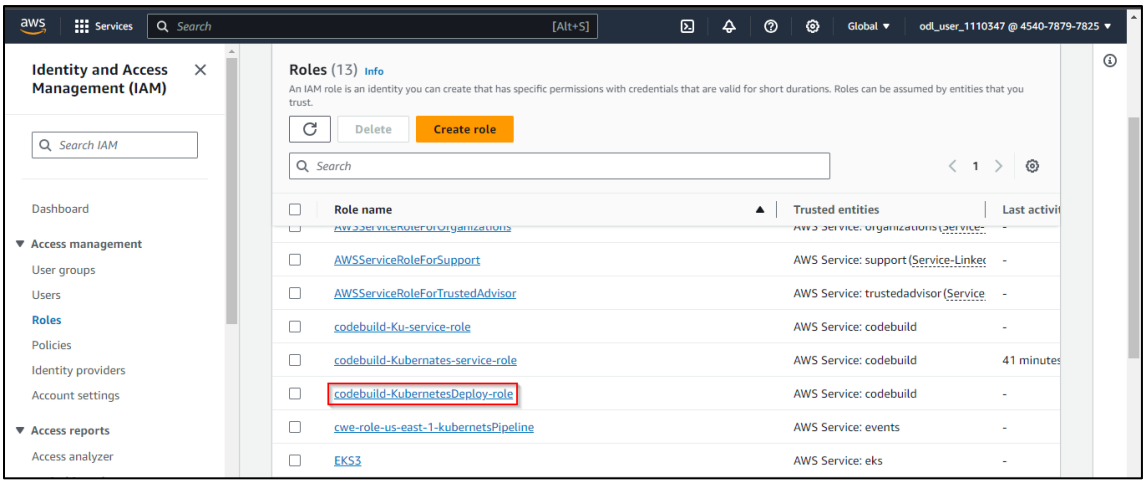
3.8 Update the following code in the **aws-auth-cm.yaml** to access API servers:

- system:node-proxier
- rolearn: arn:aws:iam::xxxaccidxxx:role/codebuild-KubernetesDeploy-service-role
- username: CodeBuild Role to Access EKS
groups:
- system:masters

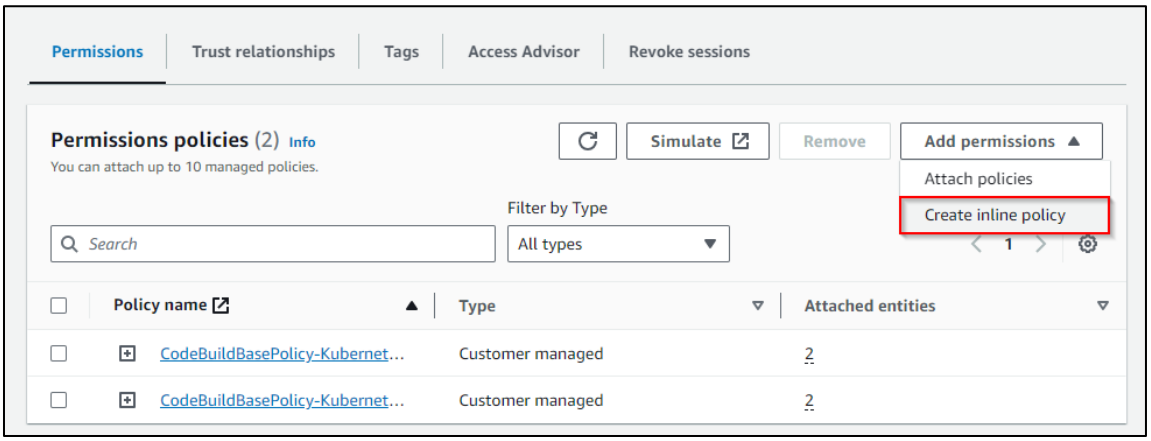


Attach a policy to the CodeBuild Service Role that allows the **eks:DescribeCluster** action. This will enable CodeBuild to download the kubeconfig file onto its server.

3.9 Navigate to the IAM Roles, and select **codebuild-KubernetesDeploy-role**

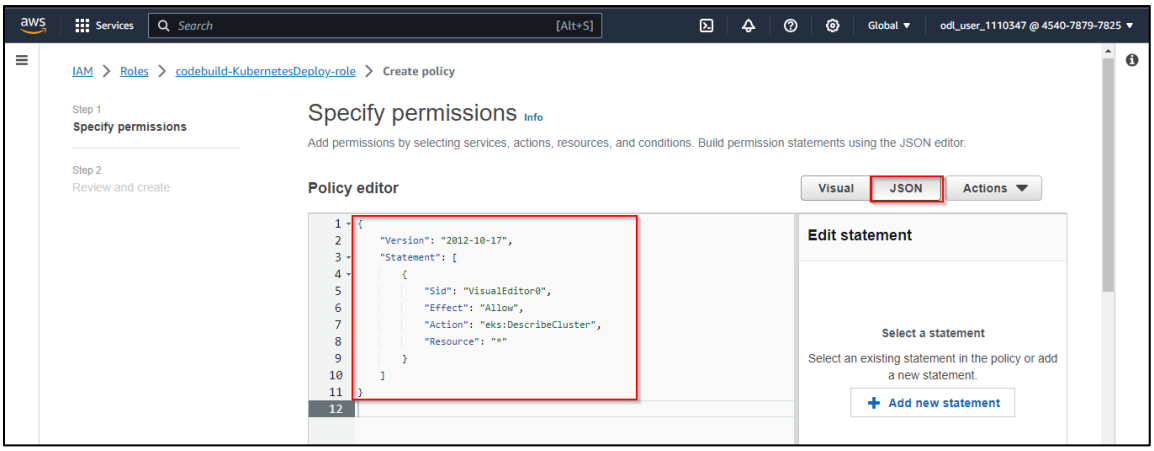


3.10 Click **Create inline policy**

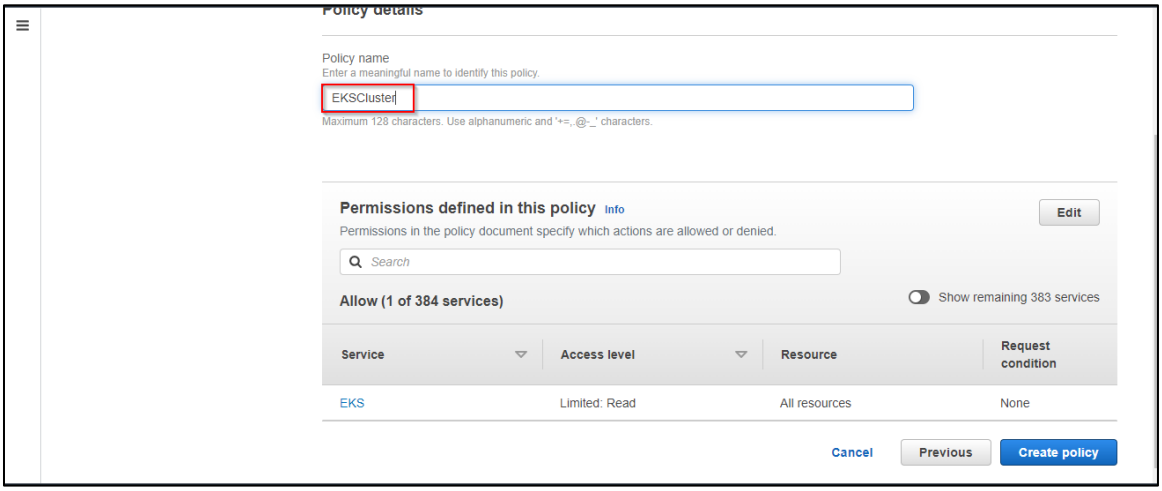


3.11 Input the following JSON policy content and click **Next**:

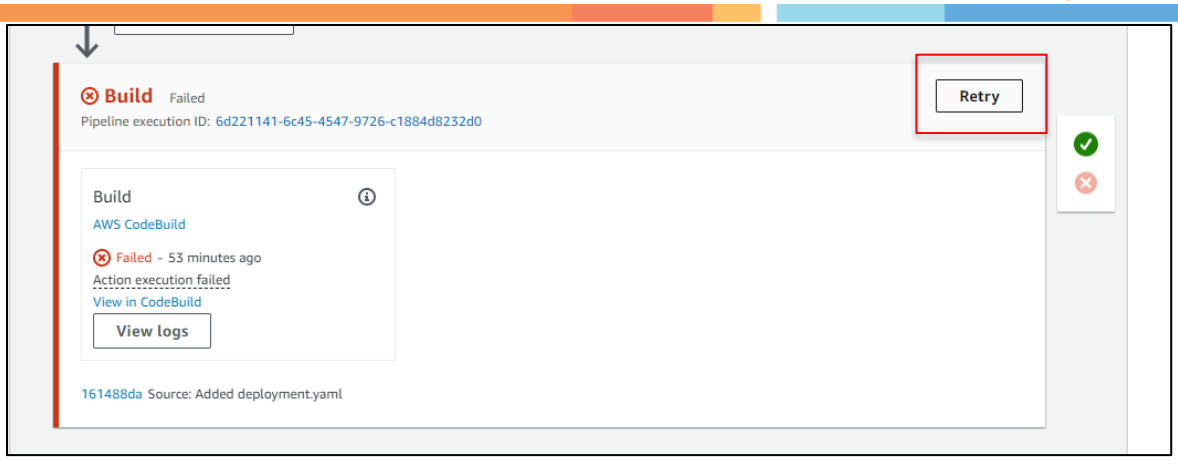
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": "eks:DescribeCluster",
      "Resource": "*"

    }

  ]
}
```



3.12 Name the policy **EKSCluster** and click **Create policy**



After modifying the IAM CodeBuild project role, you can retry the Build stage to successfully connect with the EKS Cluster and execute the YAML manifest files.



By following these steps, you have successfully set up an automated deployment process for Kubernetes applications using AWS CodePipeline.